

EDUGENIE – GOOGLE GEMINI POWERED LEARNING ASSISTANT

TESTING

1. Introduction

Testing is performed to verify that EduGenie works correctly, handles user inputs properly, communicates with the AI services, and produces the expected educational outputs.

The major features tested are:

- Question and Answer
- Concept Explanation
- Quiz Generation
- Text Summarization
- Learning Recommendations

The project uses **Pytest** for automated testing. Gemini responses can be mocked during testing, allowing tests to run without making paid API requests.

2. Testing Objectives

The main objectives are:

1. Verify that all major features work correctly.
 2. Verify FastAPI endpoints.
 3. Verify frontend and backend integration.
 4. Validate user inputs.
 5. Test error handling.
 6. Verify AI response processing.
 7. Test the application health endpoint.
 8. Identify and fix errors before final submission.
-

3. Testing Scope

Testing covers:

- Functional Testing

- Unit Testing
- API Testing
- Integration Testing
- System Testing
- Input Validation
- Error Handling
- Frontend Testing
- Regression Testing
- Basic Security Testing

4. Testing Environment

Component	Technology
Operating System	Windows / Linux / macOS
Programming	Python 3.10+
Backend	FastAPI
Server	Uvicorn
Frontend	HTML, CSS, Jinja2
AI Service	Google Gemini
Optional AI	LaMini-Flan-T5-783M
Testing	Pytest
Browser	Chrome / Edge / Firefox
IDE	Visual Studio Code

The current project documentation specifies Python 3.10+ and Pytest for testing.

5. Testing Methodology

The testing process follows:

Test Planning



Unit Testing



API Testing



Integration Testing



System Testing



Error Testing



Final Validation

6. Functional Testing

Functional testing verifies each major EduGenie feature.

Test ID	Feature	Input	Expected Result
TC01	Q&A	Academic question	AI-generated answer
TC02	Explanation	Topic + level	Concept explanation
TC03	Quiz	Educational passage	Three MCQs generated
TC04	Summary	Educational passage	Concise summary
TC05	Learning Path	Subject + level	Learning recommendations
TC06	Health Check	/health	Successful health response
TC07	API Docs	/docs	API documentation displayed

The current repository documents these five core learning endpoints along with /health and /docs.

7. Question Answering Test

Endpoint:

/qa

Test Input

Which is the largest ocean?

Expected Result

The system should return an appropriate AI-generated answer.

Status

Pass when a valid response is returned.

8. Concept Explanation Test

Endpoint:

/explain

Test Input

Topic: Pythagoras theorem

Level: Beginner

Expected Result

The system should provide an understandable explanation suitable for the requested level.

9. Quiz Generation Test

Endpoint:

/quiz

Test Input

Water cycle and its stages.

Expected Result

The system should generate **three multiple-choice questions** with answer options and correct answers.

The current implementation uses Gemini structured output for quiz generation.

10. Text Summarization Test

Endpoint:

/summarize

Test Input

An educational passage about photosynthesis.

Expected Result

The system should generate a concise summary containing the important information.

11. Learning Recommendation Test

Endpoint:

/learn/recommendations

Test Input

Subject: SQL

Level: Beginner

Expected Result

The system should generate suitable learning recommendations for a beginner.

12. Input Validation Testing

The application should be tested with invalid or incomplete inputs.

Examples:

- Empty question.
- Empty topic.
- Empty passage.
- Missing learning level.
- Missing subject.
- Invalid request data.

Expected Result

The application should return an appropriate validation or error response instead of failing unexpectedly.

13. API Testing

The following endpoints are tested individually:

/qa

/explain

/quiz

/summarize

/learn/recommendations

/health

Testing verifies:

- Request handling.
 - Input validation.
 - Response format.
 - HTTP status.
 - Error handling.
-

14. Integration Testing

Integration testing verifies communication between the major components.

Frontend

↓

FastAPI

↓

API Endpoint

↓

Gemini / Local Model

↓

Generated Response



Frontend

Expected Result

The user should be able to submit a request through the web interface and receive the generated result.

15. AI Service Testing

Gemini-based functionality is tested for:

- Valid API key.
- Invalid API key.
- API failure.
- Network failure.
- Unexpected response.
- Model configuration errors.

Expected Result

AI-related errors should be handled without crashing the complete application.

16. Local Model Testing

EduGenie optionally supports **LaMini-Flan-T5-783M** for concept explanation.

Testing includes:

1. Enable local model.
2. Submit explanation request.
3. Verify generated explanation.
4. Disable local model.
5. Verify Gemini-based explanation.

The local explanation model is optional and disabled by default in the current project.

17. Frontend Testing

The web interface is tested for:

- Page loading.
- Feature selection.
- Text input.
- Submit button.
- Backend communication.
- Result display.
- Error display.
- Repeated requests.

Expected Result

The interface should allow users to complete each learning task without unexpected errors.

18. Automated Testing

The project uses **Pytest** for automated tests.

Test command:

```
pytest -q
```

Gemini responses are mocked during automated testing so that tests can run without requiring paid Gemini API calls.

19. Security Testing

Basic security checks include:

- Verify that the Gemini API key is not hard-coded.
- Verify that .env is not committed to GitHub.
- Verify that invalid requests are handled.
- Verify that sensitive configuration is protected.
- Check request-size handling for future production deployment.

The repository recommends additional production security measures such as authentication, rate limiting, secret management, HTTPS, and request-size limits.

20. Regression Testing

Regression testing is performed after modifications to ensure existing features continue to work.

The following features should be re-tested after major changes:

- Q&A
- Explanation
- Quiz
- Summary
- Learning Recommendations
- Health Check
- API Documentation

21. Test Result Summary

Module	Test Result
Q&A	Pass
Concept Explanation	Pass
Quiz Generation	Pass
Text Summarization	Pass
Learning Recommendations	Pass
Health Check	Pass
API Documentation	Pass
Input Validation	Pass
Error Handling	Pass
Automated Tests	Pass

Note: The final status should be updated after executing the complete test suite on the final submitted project version.

22. Defect Handling

When an error is identified:

1. Record the problem.
2. Identify the affected module.
3. Reproduce the error.
4. Modify the implementation.
5. Run the relevant test.
6. Run regression tests.
7. Confirm that the issue is resolved.

This process helps prevent previously fixed issues from reappearing.

23. Final Testing Checklist

Before submission, verify:

- Application starts successfully.
- Homepage loads correctly.
- Q&A works.
- Concept explanation works.
- Quiz generation works.
- Summarization works.
- Learning recommendations work.
- /health works.
- /docs works.
- Invalid inputs are handled.
- AI errors are handled.
- Automated tests pass.
- API key is protected.
- No sensitive information is committed.

- Final documentation is complete.
-

24. Conclusion

Testing ensures that EduGenie provides reliable functionality across its major educational features.

The testing process covers **functional testing, API testing, integration testing, input validation, AI service testing, frontend testing, security checks, and automated testing.**